MASTERING .NET PROGRAMMING

UNIT III — OPERATORS, CONTROL CONSTRUCTS, CLASSES & METHODS

Beginner-friendly teaching notes • Definitions • Why • How • Examples • C# code • Practice

1. What is an Operator?

Definition: An operator is a symbol that tells C# to perform an operation on one or more values. The values involved are called operands.

```
int a = 10;
int b = 3;

int sum = a + b;
int remainder = a % b;

Console.WriteLine(sum);
Console.WriteLine(remainder);
```

Here + and % are operators. The variables a and b are operands. The result of the expression is stored in another variable.

> **Real-life analogy**
> A calculator button such as + tells the calculator what action to perform. A programming operator plays a similar role.

2. Arithmetic Operators

Arithmetic operators are used for numerical calculations.

Operator	Meaning	Example
+	Addition	10 + 3 = 13
-	Subtraction	10 - 3 = 7
*	Multiplication	10 * 3 = 30
/	Division	10 / 3 = 3 when both operands are int
%	Remainder	10 % 3 = 1

```
int a = 10;
int b = 3;

Console.WriteLine(a + b);
Console.WriteLine(a - b);
Console.WriteLine(a * b);
Console.WriteLine(a / b);
Console.WriteLine(a % b);
```

Important: integer division removes the fractional part. For 10 / 3, the integer result is 3.

```
double result = 10.0 / 3.0;
Console.WriteLine(result);
```

3. Relational Operators

Relational operators compare two values and produce a Boolean result: true or false.

Operator	Meaning
==	Equal to

Operator	Meaning
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

```
int marks = 75;

Console.WriteLine(marks >= 40); // True
Console.WriteLine(marks == 100); // False
```

Relational operators are commonly used in decision-making statements.

4. Logical Operators

Logical operators combine or modify Boolean conditions.

Operator	Meaning	Simple idea
&&	Logical AND	Both conditions must be true
||	Logical OR	At least one condition must be true
!	Logical NOT	Reverses true/false

```
int age = 22;
bool hasId = true;

if (age >= 18 && hasId)
{
Console.WriteLine("Entry allowed");
}
```

The && operator requires both age >= 18 and hasId to be true.

5. Bitwise Operators

Bitwise operators work on the binary representation of integral values. They are especially useful in low-level operations, flags and certain performance-oriented tasks.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise complement
<<	Left shift
>>	Right shift

```
int x = 5; // 0101
int y = 3; // 0011

int result = x & y;

Console.WriteLine(result); // 1
```

Only the binary bits that are 1 in both operands remain 1 for bitwise AND.

6. Special and Useful Operators

C# provides several operators that solve common programming tasks.

Operator	Purpose / example
?:	Conditional/ternary operator
??	Use a fallback when a value is null
??=	Assign a value if the variable is null
is	Check type compatibility
as	Attempt a compatible reference conversion
new	Create an object
typeof	Obtain type information

```
int age = 20;

string result = age >= 18 ? "Adult" : "Minor";

Console.WriteLine(result);
```

The conditional operator is a compact alternative to a simple if-else assignment.

7. Expressions

Definition: An expression is a combination of values, variables, operators, method calls and other elements that produces a result.

```
int price = 100;
int quantity = 3;
int total = price * quantity;
```

price * quantity is an expression. It produces 300, which is assigned to total.

```
double finalAmount = price * quantity + 50;
```

Expressions can be simple or complex. Parentheses should be used when they make the intended calculation clearer.

8. Operator Precedence

Precedence determines which operator is evaluated first when several operators appear in one expression.

```
int x = 10 + 5 * 2;
Console.WriteLine(x); // 20
```

Multiplication has higher precedence than addition, so 5 * 2 is calculated first.

```
int y = (10 + 5) * 2;
Console.WriteLine(y); // 30
```

Parentheses explicitly change the order.

> **Best practice**
> When an expression could confuse a reader, use parentheses even if you know the precedence rules.

9. Associativity

Associativity determines how operators with the same precedence are grouped. Many arithmetic operators are evaluated from left to right.

```
int result = 20 - 5 - 3;
// (20 - 5) - 3 = 12
```

Students should not try to memorize every rule at once. Use parentheses whenever clarity is important.

10. Decision Making with if

Decision statements allow a program to execute different code depending on a condition.

```
int marks = 72;
```

```
if (marks >= 40)
{
Console.WriteLine("Pass");
}
else
{
Console.WriteLine("Fail");
}
```

The condition marks >= 40 is evaluated. If it is true, the first block runs; otherwise the else block runs.

11. if-else-if Ladder

An if-else-if ladder checks multiple conditions in order.

```
int marks = 82;

if (marks >= 90)
{
Console.WriteLine("A+");
}
else if (marks >= 75)
{
Console.WriteLine("A");
}
else if (marks >= 60)
{
Console.WriteLine("B");
}
else
{
Console.WriteLine("C");
}
```

Once one condition is true, its block executes and the remaining else-if conditions are skipped.

12. Nested if

A nested if means one decision statement is placed inside another.

```
int age = 20;
bool hasId = true;

if (age >= 18)
{
if (hasId)
{
Console.WriteLine("Entry allowed");
}
}
```

Nested decisions are useful when one condition depends on another, but too much nesting can make code difficult to read.

13. switch Statement

switch is useful when one expression must be compared against multiple possible values.

```
int choice = 2;

switch (choice)
{
case 1:
Console.WriteLine("Add");
break;

case 2:
Console.WriteLine("Update");
break;

case 3:
Console.WriteLine("Delete");
break;
```

```
default:
Console.WriteLine("Invalid choice");
break;
}
```

The expression choice is compared with each case. The matching case executes. In this traditional form, break exits the switch section.

> **When to use switch?**
> It is often clearer than a long chain of equality-based if-else statements, such as menu choices or command types.

14. What is a Loop?

Definition: A loop repeatedly executes a block of code while a condition or collection rule allows it.

Without loops, printing the numbers 1 to 100 would require many repeated statements. A loop lets us describe the repetition once.

15. for Loop

A for loop is useful when the number of iterations is known or controlled by a counter.

```
for (int i = 1; i <= 5; i++)
{
Console.WriteLine(i);
}
```

Step 1: int i = 1 initializes the counter. **Step 2:** i <= 5 checks the condition. **Step 3:** the body executes. **Step 4:** i++ increases the counter. The process repeats.

> **Real-life analogy**
> Climbing five numbered steps: start at step 1, continue while the step number is at most 5, then move to the next step.

16. for Loop — Multiplication Table

```
int n = 5;

for (int i = 1; i <= 10; i++)
{
Console.WriteLine(n + " x " + i + " = " + (n * i));
}
```

This is a practical example students can run immediately.

17. while Loop

A while loop checks its condition before executing the body. It is useful when the number of repetitions is not known in advance.

```
int i = 1;

while (i <= 5)
{
Console.WriteLine(i);
i++;
}
```

If the condition is false before the first iteration, the body may execute zero times.

18. do-while Loop

A do-while loop executes its body first and checks the condition afterward. Therefore, the body executes at least once.

```
int i = 1;

do
{
```

```
Console.WriteLine(i);
i++;
}
while (i <= 5);
```

This is useful for menus where the program must display the menu at least once before asking whether to continue.

19. foreach Loop

foreach is designed to visit each element in a collection or array.

```
string[] names = { "Aman", "Ravi", "Zaid" };

foreach (string name in names)
{
Console.WriteLine(name);
}
```

The loop automatically obtains each element. You do not need to manage an index manually.

20. for vs while vs do-while vs foreach

Loop	Best suited for
for	Counter-controlled repetition
while	Repetition controlled by a condition
do-while	When the body must execute at least once
foreach	Visiting every element in a collection

21. break and continue

break immediately exits the loop. **continue** skips the remaining statements of the current iteration and moves toward the next iteration.

```
for (int i = 1; i <= 10; i++)
{
if (i == 5)
break;

Console.WriteLine(i);
}
```

This prints 1 through 4 because the loop stops when i becomes 5.

```
for (int i = 1; i <= 5; i++)
{
if (i == 3)
continue;

Console.WriteLine(i);
}
```

This skips printing 3 but continues with 4 and 5.

22. What is a Class?

Definition: A class is a blueprint or definition that describes the data and behavior associated with objects.

A class can contain fields, properties, methods, constructors and other members.

```
class Student
{
public string Name;
public int Age;

public void Display()
{
Console.WriteLine(Name + " " + Age);
}
```

```
}
```

Student is a type. It describes what information a student object can contain and what actions it can perform.

23. What is an Object?

Definition: An object is an instance created from a class.

```
Student s = new Student();

s.Name = "Aman";
s.Age = 20;

s.Display();
```

new Student() creates the object. s is the variable that refers to that object.

> **Class vs object**
> Class = blueprint. Object = actual instance. For example, Student is the class; s is one Student object.

24. Methods

Definition: A method is a named block of code that performs a task. It can receive parameters and may return a value.

```
static int Add(int a, int b)
{
return a + b;
}

int result = Add(10, 20);
Console.WriteLine(result);
```

Here Add has two parameters and returns an int. The method call Add(10, 20) supplies the arguments.

Part	Meaning
Return type	Type of value returned, or void if nothing is returned.
Method name	Identifier used to call the method.
Parameters	Inputs declared by the method.
Body	Statements that perform the task.

25. Method with void

```
static void DisplayMessage()
{
Console.WriteLine("Welcome");
}

DisplayMessage();
```

void means the method does not return a value.

26. Constructors

Definition: A constructor is a special member used to initialize an object when it is created.

```
class Student
{
public string Name;

public Student(string name)
{
Name = name;
}
}

Student s = new Student("Aman");
```

```
Console.WriteLine(s.Name);
```

When new Student("Aman") executes, the constructor receives the value and initializes Name.

27. Parameterless and Parameterized Constructors

```
class Student
{
public string Name;

public Student()
{
Name = "Unknown";
}

public Student(string name)
{
Name = name;
}
}
```

The class has two constructors. This is an example of constructor overloading.

28. Destructors / Finalizers

In C#, the term destructor is used for a **finalizer**. It is associated with cleanup that occurs when an object is finalized by the garbage collector.

```
class Demo
{
~Demo()
{
// finalizer
}
}
```

Finalizers are not deterministic: you cannot rely on them to run at a particular time. For resources such as files, streams and handles, IDisposable and using are preferred.

> **Important**
> Do not use a finalizer as a normal replacement for Dispose(). Managed memory is handled by the garbage collector, while external resources often need deterministic cleanup.

29. Method Overloading

Definition: Method overloading means having multiple methods with the same name but different parameter lists.

```
class Calculator
{
public int Add(int a, int b)
{
return a + b;
}

public double Add(double a, double b)
{
return a + b;
}

public int Add(int a, int b, int c)
{
return a + b + c;
}
}
```

The compiler chooses the appropriate overload based on the arguments used in the method call.

```
Calculator c = new Calculator();

Console.WriteLine(c.Add(10, 20));
```

```
Console.WriteLine(c.Add(10.5, 20.5));
Console.WriteLine(c.Add(1, 2, 3));
```

Remember
Overloading changes the parameter list. It does not require changing the method name.

30. Operator Overloading

Definition: Operator overloading allows selected operators to have meaningful behavior for user-defined types.

```
class Point
{
public int X;
public int Y;

public static Point operator +(Point a, Point b)
{
return new Point
{
X = a.X + b.X,
Y = a.Y + b.Y
};
}
}

Point p1 = new Point { X = 2, Y = 3 };
Point p2 = new Point { X = 4, Y = 5 };

Point p3 = p1 + p2;

Console.WriteLine(p3.X);
Console.WriteLine(p3.Y);
```

The overloaded + operator makes p1 + p2 meaningful for Point objects.

31. Complete Beginner Program — Student Result

```
using System;

class Student
{
public string Name;
public int Marks;

public Student(string name, int marks)
{
Name = name;
Marks = marks;
}

public void DisplayResult()
{
if (Marks >= 40)
Console.WriteLine(Name + ": Pass");
else
Console.WriteLine(Name + ": Fail");
}
}

class Program
{
static void Main()
{
Student s = new Student("Aman", 75);
s.DisplayResult();
}
}
```

Concepts combined: class, object, constructor, fields, method, parameter, if-else and object creation.

32. Common Beginner Confusions

• **= vs ==:** = assigns a value; == compares two values.

• **while vs do-while:** while may execute zero times; do-while executes the body at least once.

• **method vs constructor:** a method performs an operation; a constructor initializes an object.

• **overloading vs overriding:** this unit covers overloading; overriding belongs to inheritance/polymorphism in Unit IV.

• **destructor vs Dispose:** a C# destructor is a finalizer; IDisposable/Dispose is the normal deterministic resource-cleanup pattern.

• **integer division:** 10 / 3 with int operands produces 3, not 3.333...

33. Unit III Quick Revision

Question	One-line answer
What is an operator?	A symbol that performs an operation on operands.
What is an expression?	A combination of elements that produces a value/result.
What is precedence?	The rule determining which operators are evaluated first.
What is if-else?	A decision-making construct.
What is switch?	A multi-case selection construct.
What is a loop?	A construct that repeats code.
What is a class?	A blueprint/definition for objects.
What is an object?	An instance of a class.
What is a constructor?	A member used to initialize an object during creation.
What is overloading?	Multiple methods/operators with different signatures/parameter forms.

34. Practice Questions

• Explain arithmetic, relational, logical and bitwise operators with examples.

• Explain operator precedence and associativity.

• Write a C# program to check whether a number is even or odd.

• Write a program using switch to create a simple menu.

• Explain for, while, do-while and foreach with examples.

• Write a C# program to print a multiplication table using for.

• Define class and object with a program.

• Explain methods, parameters and return values.

• Explain constructors with parameterless and parameterized examples.

• What is method overloading? Write a program.

• What is operator overloading? Demonstrate it using a Point class.

• Explain destructors/finalizers and why IDisposable is preferred for resource cleanup.

35. Teaching Checklist

• Begin with arithmetic operators students already know from mathematics.

• Run the even/odd and multiplication-table programs before introducing complex expressions.

• Draw a simple flow showing condition → decision → selected block.

• Explain loops using repetition students can see, such as printing 1–10 and multiplication tables.

• Introduce class as a blueprint and object as an actual instance before teaching constructors.

• Demonstrate method parameters and return values separately.

• Teach overloading only after students understand normal methods.

• Clarify that overriding belongs to Unit IV and is different from overloading.